Lecture 06 — LQR, iLQR, MPC

Reinforcement Learning 2026–27

Lecture notes based on the slide deck (slides 1–93) and the companion notes "Finite-horizon MDPs and optimal control" and "LQR, iLQR, MPC"

Abstract

These notes follow the *order of the slides*: each section/subsection carries the (near-verbatim) slide title and, in parentheses, the slide range. The formal statements and proofs of the two companion PDFs are merged at the point where the slides introduce the topic. After each formal result there is a blue *Intuition* box. Material that appears only in the PDFs and has no slide is collected in Appendix A. Where a figure of the slides could not be redrawn (animated GIFs), a red dashed placeholder says which image to paste and where it comes from.

Contents

1 Recap *(slides 2–10)* **3**
- 1.1 Policy Iteration *(slides 3–6)* . . . 3
- 1.2 We Did Dynamic Programming! *(slides 7–8)* . . . 3
- 1.3 Primal Linear Program *(slides 9)* . . . 4

2 Finite-Horizon MDPs *(slides 11–25)* **4**
- 2.1 Definition and time-dependent policies *(slides 11–15)* . . . 4
- 2.2 Finite-Horizon MDPs — Example *(slides 16)* . . . 4
- 2.3 Finite-Horizon MDP & Policy Interaction *(slides 17–21)* . . . 5
- 2.4 Finite-Horizon MDP: V & Q *(slides 22–24)* . . . 5
- 2.5 Finite-Horizon MDP: Bellman Equation *(slides 25)* . . . 5

3 Finding the Optimal Policy *(slides 26–32)* **5**

4 Control Problems and Optimal Control *(slides 33–39)* **6**
- 4.1 Control Problems *(slides 33–36)* . . . 6
- 4.2 Optimal Control *(slides 37–39)* . . . 7

5 Discretization and Bellman's Curse of Dimensionality *(slides 40–42)* (SKIPPED IN CLASS) **7**

6 Linear Systems and Quadratic Cost *(slides 43–46)* **8**
- 6.1 Linear Systems *(slides 43–44)* . . . 8
- 6.2 Quadratic Cost Function *(slides 45–46)* . . . 8

7 Q Functions for Linear Systems and Quadratic Cost *(slides 47–54)* **9**

8 Finding the Best Action and the Inductive Step *(slides 55–63)* **9**
- 8.1 Finding the Best Action *(slides 55–57)* . . . 9
- 8.2 Inductive Step: Proof *(slides 58–63)* . . . 10

9 The LQR Algorithm *(slides 64–70)* **10**
- 9.1 Cost at H *(slides 64–65)* . . . 10
- 9.2 LQR algorithm *(slides 66–70)* . . . 10

10 LQR Extensions *(slides 71–76)* **12**
10.1 LQR for Linear Time Varying (LTV) Systems *(slides 72–73)* . . . 12
10.2 LQR for Affine Systems *(slides 74)* . . . 12
10.3 LQR for Stochastic Systems *(slides 75)* . . . 12
10.4 Penalize for Change in Controls *(slides 76)* . . . 13

11 LQR for Non-Linear Systems *(slides 77–82)* **13**

12 LQR for Trajectory Following in Non-Linear Systems *(slides 83–85)* **14**

13 Iterative LQR (iLQR) *(slides 86–92)* **15**
13.1 Iterative LQR (iLQR) *(slides 86)* . . . 15
13.2 Iterative LQR — Algorithm *(slides 87)* . . . 15
13.3 Iterative LQR — LTV *(slides 88–89)* . . . 15
13.4 Iterative LQR — Convergence *(slides 90–91)* . . . 16
13.5 Iterative LQR — Tricks *(slides 92)* . . . 17

14 Receding Horizon Control, i.e. Model Predictive Control (MPC) *(slides 93)* **17**

15 Summary **18**
15.1 Comparison of the methods . . . 18
15.2 Key take-aways . . . 18

A Supplementary material (in the companion PDFs, not on the slides) **19**

1 Recap *(slides 2–10)*

The deck opens with a recap of the dynamic-programming algorithms for (infinite-horizon, discounted) MDPs, to set the stage for the control setting.

1.1 Policy Iteration *(slides 3–6)*

Policy iteration outputs a *policy* at every iteration, $\{\pi_0, \pi_1, \pi_2, \ldots, \pi_T\}$, whereas value iteration outputs *values*. The procedure is:

1. start with a random guess π_0 (deterministic or stochastic);
2. for $t = 0, \ldots, T$: do *policy evaluation*, i.e. compute $Q^{\pi_t}(s, a)$ for all (s, a); then *policy improvement*, $\pi_{t+1}(s) \in \arg\max_a Q^{\pi_t}(s, a)$.

The algorithm only makes progress: the performance of the policy is monotonically non-decreasing.

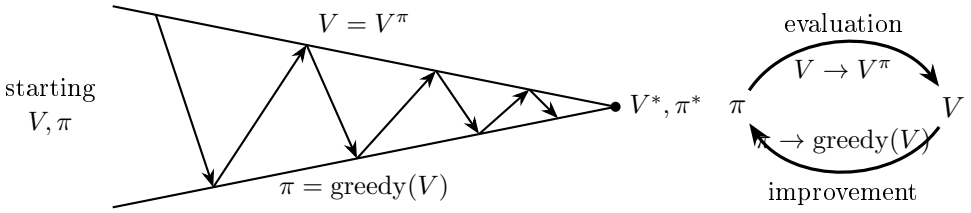


Figure 1: Redrawn from slide 4 (credits on the slide: David Silver). Left: policy iteration alternates two projections and spirals into (V^*, π^*). Right: the evaluation/improvement cycle.

Properties (slides 5–6). Is there a maximum number of iterations? Yes: $|A|^{|S|}$, the number of deterministic policies. Since each improvement step is monotone, a policy can appear in at most one round unless it is already optimal. *Stopping rule:* stop when the policy does not change any more in any state.

Intuition

Policy iteration is a "coordinate ascent" between two objects: given a policy, you can compute exactly how good it is (evaluation); given values, you can read off a policy that is at least as good (greedy improvement). Because each round can only help, and there are finitely many policies, you cannot cycle: you must land on the optimum.

1.2 We Did Dynamic Programming! *(slides 7–8)*

Dynamic programming applies when there is (i) **optimal substructure** (the optimal solution decomposes into subproblems) and (ii) **overlapping subproblems** (subproblems recur and their solutions can be cached). MDPs have both, thanks to the Bellman equation. We used DP for *planning*: we assumed the transition probabilities of the MDP are known.

Problem	Bellman equation	Algorithm
Prediction	Bellman expectation equation	Iterative policy evaluation
Control	Bellman expectation eq. + greedy improvement	Policy iteration
Control	Bellman optimality equation	Value iteration

Table 1: Table of slide 8 (credits: David Silver), retyped.

1.3 Primal Linear Program *(slides 9)*

As an alternative to VI and PI, the Bellman optimality equation can be written as a linear program: minimise $\sum_s V(s)$ subject to the Bellman inequalities $V(s) \geq r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V(s')$ for all (s,a). The optimal V^* is the pointwise smallest feasible vector, hence the unique minimiser. (The slide shows the objective "$\min V(s)$" and the constraints as images; this is the standard form.)

Intuition

Any V that satisfies all Bellman inequalities is an *upper bound* on what any policy can achieve; the best upper bound that is still valid is V^* itself. A linear program finds it without iterating Bellman updates.

2 Finite-Horizon MDPs *(slides 11–25)*

2.1 Definition and time-dependent policies *(slides 11–15)*

The deck uses a slightly different formulation: (S, A, R, T, H, μ_0), with horizon H and initial-state distribution $s_0 \sim \mu_0$. (The companion notes write the same object as (S, A, p, r, H, μ), $H \geq 1$.)

Definition 2.1 (Finite-horizon MDP)**.** A finite-horizon MDP is a tuple (S, A, p, r, H, μ) with horizon $H \geq 1$ and initial distribution $s_0 \sim \mu$. An *episode* is $s_0, a_0, s_1, \ldots, s_{H-1}, a_{H-1}$, and there is *no discount factor*. Policies are *time-dependent*: $\pi = (\pi_0, \pi_1, \ldots, \pi_{H-1})$ with $a_h = \pi_h(s_h)$.

The slides stress (slides 13–15): the action may differ for the same state depending on t (e.g. "I could explore more at the beginning"), and this is *very common in control.*

Intuition

With a deadline, the *time remaining* is part of the situation. Think of an exam: with two hours left you explore a hard problem; with two minutes left you write down what you have. The same state ("this question is open") calls for different actions. A stationary policy cannot express that; a time-dependent one can. A concrete numerical example where this is *necessary* is in Appendix A.

2.2 Finite-Horizon MDPs — Example *(slides 16)*

The slide shows a wheeled robot that must follow a reference trajectory: state $(X(t), Y(t))$ (plus orientation), controls v (forward speed) and ω (turn rate), reference point $(X_r(t), Y_r(t))$. The cost penalises the distance to the moving reference over a finite horizon, which is a natural finite-horizon problem.

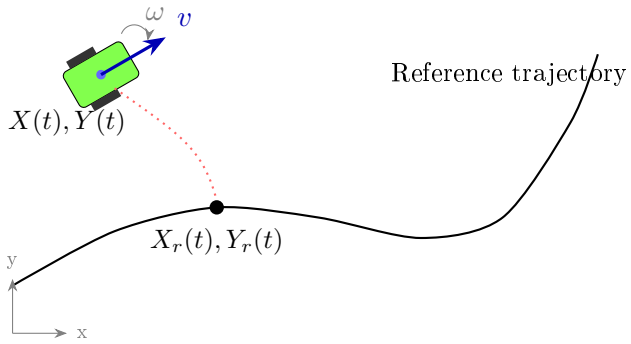
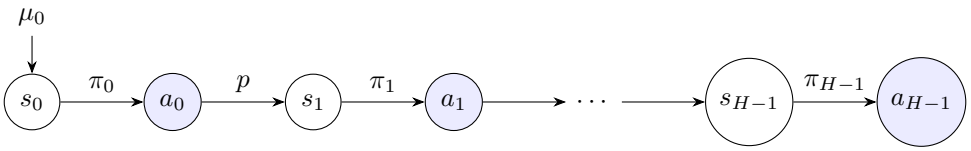


Figure 2: Redrawn from the example on slide 16: a robot with state (X, Y) and controls (v, ω) tracking a moving reference point.

2.3 Finite-Horizon MDP & Policy Interaction *(slides 17–21)*

The slides build the interaction step by step: $s_0 \sim \mu_0$; the policy picks a_0; the environment draws $s_1 \sim P(\cdot \mid s_0, a_0)$; the policy picks a_1; …; until the episode $(s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1})$ is complete.



2.4 Finite-Horizon MDP: V & Q *(slides 22–24)*

With no discount factor, for $s_h = s$, $a_\tau = \pi_\tau(s_\tau)$ and $s_{\tau+1} \sim P(\cdot \mid s_\tau, a_\tau)$:

$$V_h^\pi(s) = \mathbb{E}_\pi\Big[\sum_{\tau=h}^{H-1} r(s_\tau, a_\tau)\Big], \qquad Q_h^\pi(s,a) = r(s,a) + \mathbb{E}_{s'\sim p(\cdot|s,a)}\big[V_{h+1}^\pi(s')\big],$$

where in Q_h^π the first action is $a_h = a$ and afterwards π is followed. The optimal policy is the collection $\pi^* = \{\pi_0^*, \pi_1^*, \ldots, \pi_{H-1}^*\}$ (slide 26).

2.5 Finite-Horizon MDP: Bellman Equation *(slides 25)*

The Bellman equation links consecutive levels: $V_h^\pi(s) = Q_h^\pi(s, \pi_h(s))$ and $Q_h^\pi(s,a) = r(s,a) + \mathbb{E}_{s'}[V_{h+1}^\pi(s')]$, i.e. level h depends on level $h+1$ *only*.

Intuition

V_h is the "reward-to-go" with $H - h$ steps left. The Bellman equation says: reward-to-go = reward now + average reward-to-go from wherever you land. Level $h+1$ has already been paid for, so you only have to think one step ahead. This is precisely the structure that lets you compute everything by *walking backwards from the deadline*.

3 Finding the Optimal Policy *(slides 26–32)*

The slides note: this is an *easier problem than the infinite-horizon one*. We reason backwards in time and apply dynamic programming. At the last step $H-1$ there is no future, so $Q_{H-1}^*(s,a) = r(s,a)$; then we ask what happens at $H-2$, and so on (slides 27–32).

Algorithm 3.1 (Backward induction / finite-horizon value iteration). 1. At the last step: $Q^*_{H-1}(s,a) = r(s,a)$, $\pi^*_{H-1}(s) \in \arg\max_a Q^*_{H-1}(s,a)$, $V^*_{H-1}(s) = \max_a Q^*_{H-1}(s,a)$.
2. For $h = H-2, H-3, \ldots, 0$:

$$Q^*_h(s,a) = r(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}\left[V^*_{h+1}(s')\right],$$
$$\pi^*_h(s) \in \arg\max_a Q^*_h(s,a), \qquad V^*_h(s) = \max_a Q^*_h(s,a).$$

Theorem 3.2 (Backward induction is exact (SKIPPED IN CLASS)). *Algorithm 3.1 returns $V^*_h = \max_\pi V^\pi_h$ for every h and every state, and $\pi^* = (\pi^*_0, \ldots, \pi^*_{H-1})$ is optimal. The cost is $O(H|S|^2|A|)$ and there is no approximation error at any step.*

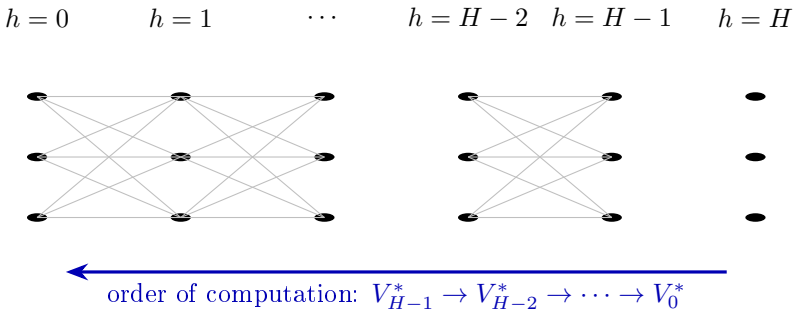
Proof. Backward induction on h. At $h = H-1$ no future remains, so $V^\pi_{H-1}(s) = r(s, \pi_{H-1}(s))$ for any π, and taking the maximum over a is optimal by inspection; the maximum is attained because A is finite.

Suppose V^*_{h+1} is the optimal value at level $h+1$. For any policy π, by the Bellman equation of the previous section,

$$V^\pi_h(s) = r(s,\pi_h(s)) + \mathbb{E}_{s'}[V^\pi_{h+1}(s')] \le r(s,\pi_h(s)) + \mathbb{E}_{s'}[V^*_{h+1}(s')] \le \max_a \left[r(s,a) + \mathbb{E}_{s'}[V^*_{h+1}(s')]\right],$$

the first inequality by the induction hypothesis and the second because the maximum dominates any particular action. The right-hand side is exactly $V^*_h(s)$ as computed by the algorithm, and it is attained by the policy that plays π^*_h now and behaves optimally afterwards, which exists by the induction hypothesis. Hence V^*_h is the optimal value and π^*_h attains it.

Cost. Each of the H levels evaluates $|S||A|$ pairs, each requiring an expectation over $|S|$ successors. □



Intuition

It is like computing the cheapest route backwards from the destination. At the end there is nothing to plan: pick the best immediate reward. One step earlier, the only thing you need to know about the future is the value table you just built — not the many futures that lead to it. Each stage costs one sweep, so the total is linear in the horizon, and nothing is approximated. That is why the finite-horizon problem is *easier*: no fixed-point equation, just one pass.

4 Control Problems and Optimal Control *(slides 33–39)*

4.1 Control Problems *(slides 33–36)*

So far the state and action spaces were discrete. What about a cartpole? State: angular position & velocity, linear position & velocity; action/control: the force on the cart; goal: find the control

policy that minimises the long-term cost. More generally in control $x \in \mathbb{R}^d$ and $u \in \mathbb{R}^k$; we write x **for the state and** u **for the action**, and we talk about **cost** rather than reward: we *minimise* cost.

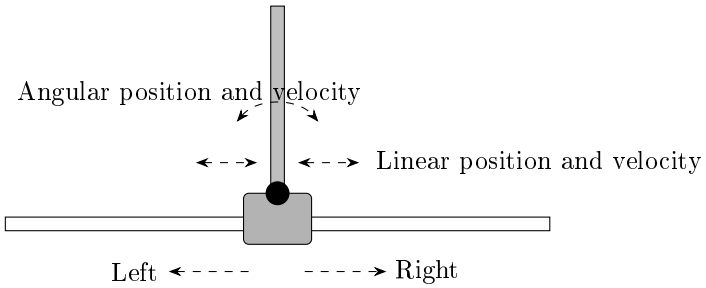


Figure 3: Cartpole, redrawn from slides 33–34: four state variables, one scalar control (horizontal force).

4.2 Optimal Control *(slides 37–39)*

Definition 4.1 (Optimal control problem)**.** Given a dynamical system with non-linear transition function f, state $x \in \mathbb{R}^d$ and control $u \in \mathbb{R}^k$, find a control policy π such that

$$\min_{\pi} \ \mathbb{E}_{\pi}\Big[c_H(x_H) + \sum_{h=0}^{H-1} c_h(x_h, u_h)\Big], \qquad u_h = \pi(x_h),\ x_0 \sim \mu_0.$$

This looks very familiar: can we treat it as a finite-horizon MDP and use value iteration? **Approach 1: discretise.**

5 Discretization and Bellman's Curse of Dimensionality *(slides 40–42)* (SKIPPED IN CLASS)

Discretise each coordinate of $x \in \mathbb{R}^d$ and $u \in \mathbb{R}^k$ at resolution ϵ. The number of grid points grows exponentially with the dimension (slide 40): $(1/\epsilon)^d$ states and $(1/\epsilon)^k$ actions. In practice discretisation is only computationally feasible up to **5 or 6 dimensional** state spaces (slides 41–42). **Approach 2:** work directly in continuous space, starting from simplified problems.

Proposition 5.1 (Bellman's curse of dimensionality)**.** *Discretising each dimension of x and u at resolution ϵ gives $|S| = \Theta((1/\epsilon)^d)$ and $|A| = \Theta((1/\epsilon)^k)$, so one level of backward induction costs $\Theta((1/\epsilon)^{2d+k})$ operations with a dense transition table, and $\Theta((1/\epsilon)^{d+k})$ when the dynamics are deterministic and each state-action pair has a single successor cell.*

Proof. A hypercube of side 1 discretised at resolution ϵ has $1/\epsilon$ points per dimension. The sweep cost of Theorem 3.2 is $|S|^2|A|$ per level, because the expectation over s' has $|S|$ terms; with a single successor it has one, and the cost is $|S||A|$. □

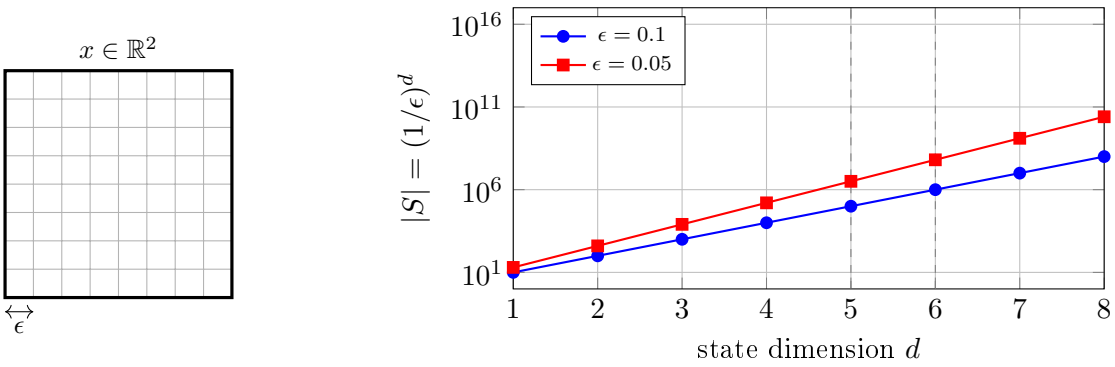


Figure 4: Left: grid at resolution ϵ (slide 40). Right: original illustration of the exponential growth; the dashed lines mark the 5–6 dimensions that the slides give as the practical limit.

Intuition

Halving ϵ multiplies the number of cells by 2^d. A 6-dimensional state at $\epsilon = 0.05$ already has 6.4×10^7 cells, and with a dense transition table the sweep costs the *square* of that times $|A|$. (The slide writes the grid size as a sum $(1/\epsilon)^d + (1/\epsilon)^k$; the number of state-action *pairs* that the sweep must touch is the product $(1/\epsilon)^{d+k}$, which is what Proposition 5.1 counts.) Discretisation is not wrong — it is just *exponentially expensive*, which motivates looking for problems where the continuous solution has a closed form.

6 Linear Systems and Quadratic Cost *(slides 43–46)*

6.1 Linear Systems *(slides 43–44)*

Consider the system

$$x_{t+1} = F_x x_t + F_u u_t, \qquad F_x \in \mathbb{R}^{d \times d},\ F_u \in \mathbb{R}^{d \times k},$$

where x_t is the state and u_t the control (action) at time t. This *is* our transition function.

6.2 Quadratic Cost Function *(slides 45–46)*

Definition 6.1 (Linear-quadratic regulator (LQR))**.** $x_{t+1} = F_x x_t + F_u u_t$, $c(x, u) = x^\top C_x x + u^\top C_u u$, with $C_x \succeq 0$ symmetric and $C_u \succ 0$ symmetric. The problem is to minimise $\mathbb{E}\big[x_H^\top C_x x_H + \sum_{h=0}^{H-1} c(x_h, u_h)\big]$.

(The slides ask for C_x, C_u both positive definite, so that any non-zero state costs something even with all-zero control; the companion notes only require $C_x \succeq 0$, which is what the proof needs.) The slides also use the alternative notation $g(x_t, u_t)$ for the cost.

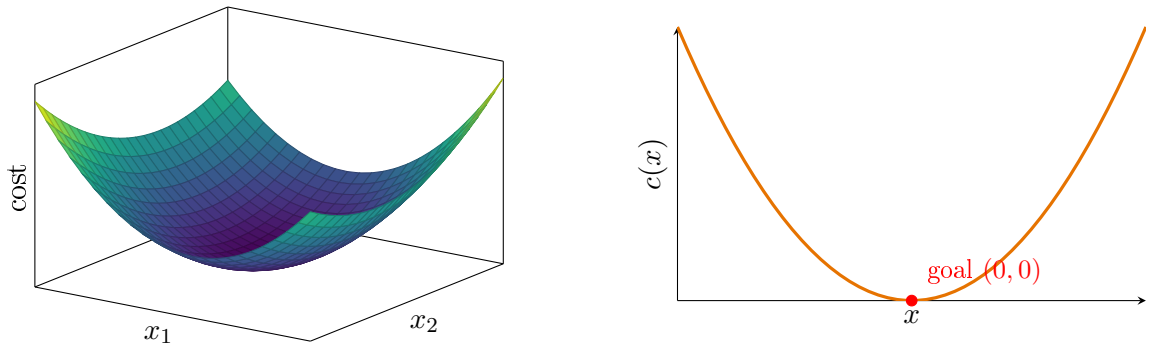


Figure 5: A quadratic cost with $C_x \succ 0$ is a bowl (two-dimensional case, left; cross-section, right), as drawn on slide 46. Its unique minimum is the goal $x = 0$.

Intuition

The cost is a bowl whose bottom is the origin: "be at the goal, and don't waste control effort". C_x says how much you dislike each kind of deviation, C_u how much you dislike each kind of effort. Note the problem is a *regulator*: the goal is to drive x to 0. All other cases (non-zero targets, trajectories) are reduced to this later by changing variables.

7 Q Functions for Linear Systems and Quadratic Cost *(slides 47–54)*

(Credits on the slides: Pieter Abbeel.) We do value iteration backwards in time, as in the previous section, but now in costs. The Q function at level h is the immediate cost plus the expected optimal cost-to-go of the next state (slides 47–49):

$$Q_h^*(x_t, u_t) = x_t^\top C_x x_t + u_t^\top C_u u_t + \mathbb{E}_{x' \sim p(\cdot|x_t, u_t)}\left[V_{h+1}^*(x')\right].$$

Deterministic for now (slide 50), the expectation disappears (slide 51):

$$Q_h^*(x_t, u_t) = x_t^\top C_x x_t + u_t^\top C_u u_t + V_{h+1}^*(F_x x_t + F_u u_t).$$

Inductive step: assumption (slide 52). Assume $V_{h+1}^*(x_t) = x_t^\top P_{h+1} x_t$ with $P \in \mathbb{R}^{d \times d}$. **Question (slide 53):** can we show that also $V_h^*(x_t) = x_t^\top P_h x_t$? **Implementing the assumption (slide 54):**

$$Q_h^*(x_t, u_t) = x_t^\top C_x x_t + u_t^\top C_u u_t + (F_x x_t + F_u u_t)^\top P_{h+1}(F_x x_t + F_u u_t).$$

Intuition

The hope is a *closed under iteration* family: if the cost-to-go is a quadratic form at level $h+1$ and the dynamics are linear, then plugging a linear map into a quadratic gives a quadratic; adding the quadratic stage cost keeps it quadratic. All that remains is the minimisation over u, and the minimum of a quadratic is easy.

8 Finding the Best Action and the Inductive Step *(slides 55–63)*

8.1 Finding the Best Action *(slides 55–57)*

We want $\pi_h^*(x) = \arg\min_u Q_h^*(x, u)$. How do you find minima? Set $\nabla_u Q_h^*(x, u) = 0$ and solve for u (slide 57).

8.2 Inductive Step: Proof *(slides 58–63)*

Everything is assembled in the following theorem, with the proof of the companion notes laid out along the slide steps.

Theorem 8.1 (LQR: exact solution by backward induction). *Define* $P_H = C_x$ *and, for* $h = H-1, \dots, 0$,

$$K_h = -\big(C_u + F_u^\top P_{h+1} F_u\big)^{-1} F_u^\top P_{h+1} F_x,$$
$$P_h = C_x + K_h^\top C_u K_h + (F_x + F_u K_h)^\top P_{h+1}(F_x + F_u K_h).$$

Then $V_h^*(x) = x^\top P_h x$ *and the optimal control is the linear feedback* $\pi_h^*(x) = K_h x$.

Proof. Backward induction on h, with the statement "V_h^* is a quadratic form $x^\top P_h x$" as hypothesis.

Base (slides 64–65). At $h = H$ no control remains and the terminal cost is $x^\top C_x x$, so $P_H = C_x$. (The slides also allow $P_H = 0$ or $C_{x,\text{final}}$, i.e. a different terminal cost.)

Step (slides 58–63). Assume $V_{h+1}^*(x) = x^\top P_{h+1} x$. Then

$$Q_h^*(x,u) = x^\top C_x x + u^\top C_u u + (F_x x + F_u u)^\top P_{h+1}(F_x x + F_u u),$$

a quadratic in (x,u) jointly (slides 59–60). Its gradient in u is

$$\nabla_u Q_h^*(x,u) = 2C_u u + 2F_u^\top P_{h+1}(F_x x + F_u u) = 2\big(C_u + F_u^\top P_{h+1} F_u\big)u + 2F_u^\top P_{h+1} F_x x.$$

The matrix $C_u + F_u^\top P_{h+1} F_u$ is positive definite: $C_u \succ 0$ by hypothesis and the second term is positive semi-definite provided $P_{h+1} \succeq 0$, which holds by induction since P_h is a sum of positive semi-definite terms. Hence Q_h^* is strictly convex in u, the stationary point is the unique minimum, and setting the gradient to zero gives $u = K_h x$ with K_h as stated (slide 61).

Substituting $u = K_h x$ back, $V_h^*(x) = Q_h^*(x, K_h x)$ (slides 62–63), i.e.

$$V_h^*(x) = x^\top C_x x + x^\top K_h^\top C_u K_h x + \big((F_x + F_u K_h)x\big)^\top P_{h+1}\big((F_x + F_u K_h)x\big) = x^\top P_h x$$

with P_h as stated, which is again a quadratic form, closing the induction. □

Intuition

The two facts that make everything work: (i) *quadratic in, quadratic out* — the cost-to-go stays a bowl at every stage, so you never leave a finite-dimensional family; (ii) the best action is a *linear* function of the state because setting a gradient of a quadratic to zero is a linear equation. The solution is a feedback gain matrix K_h ("if you are this far from the goal, push this hard in the opposite direction"). Numerical sanity check in one dimension: $x_{t+1} = x_t + u_t$, $C_x = C_u = 1$, $P_{h+1} = 1$. Then $K_h = -(1+1)^{-1} \cdot 1 \cdot 1 = -\frac{1}{2}$ and $P_h = 1 + \frac{1}{4} + (1 - \frac{1}{2})^2 \cdot 1 = 1.5$: the controller cancels half the state per step and the value goes from 1 to 1.5.

9 The LQR Algorithm *(slides 64–70)*

9.1 Cost at H *(slides 64–65)*

Recall the goal $\min \mathbb{E}_\pi[c_H(x_H) + \sum_{h=0}^{H-1} c_h(x_h, u_h)]$. At H only $c_H(x_H)$ remains, typically 0 or $x_H^\top C_x x_H$. This is the base case: $P_H = 0$ or C_x (or $C_{x,\text{final}}$).

9.2 LQR algorithm *(slides 66–70)*

Algorithm 9.1 (LQR). Initialise P_H (at 0 or C_x). Starting from $h = H-1$, *backwards*:

1. $K_h^* = -(C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x$;
2. control $u = \pi_h^*(x_t) = K_h^* x_t$;
3. $P_h = C_x + K_h^{*\top} C_u K_h^* + (F_x + F_u K_h^*)^\top P_{h+1}(F_x + F_u K_h^*)$;
4. $V_h^* = x_t^\top P_h x_t$.

Slide 67: this is the *value iteration update done in closed form*: it is always the same and solves this particular continuous-state system with quadratic cost. Slide 68: the recursion for P_h is the **Riccati equation**. Slide 69: for this kind of problem, V is renamed J, the "cost-to-go", so $J_h^* = x_t^\top P_h x_t$.

Remark. Expanding the square gives the more familiar form $P_h = C_x + F_x^\top P_{h+1} F_x - F_x^\top P_{h+1} F_u (C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x$.

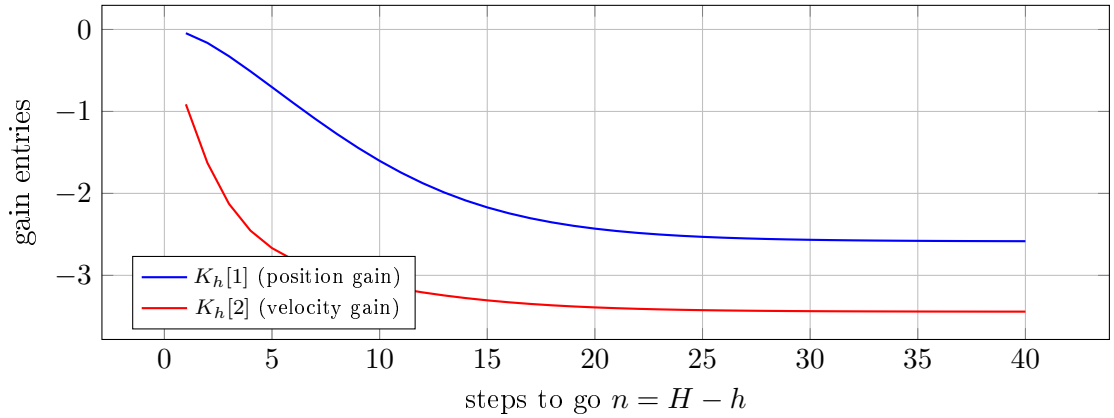


Figure 6: Original illustration (not on the slides). Riccati iteration for a discretised double integrator, $F_x = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}$, $F_u = \begin{bmatrix} 0.005 \\ 0.1 \end{bmatrix}$, $C_x = I$, $C_u = 0.1$, $P_H = C_x$, $H = 40$. Far from the end of the episode the gains change very slowly, near the end (n small) they shrink to zero: there is little time left to pay off.

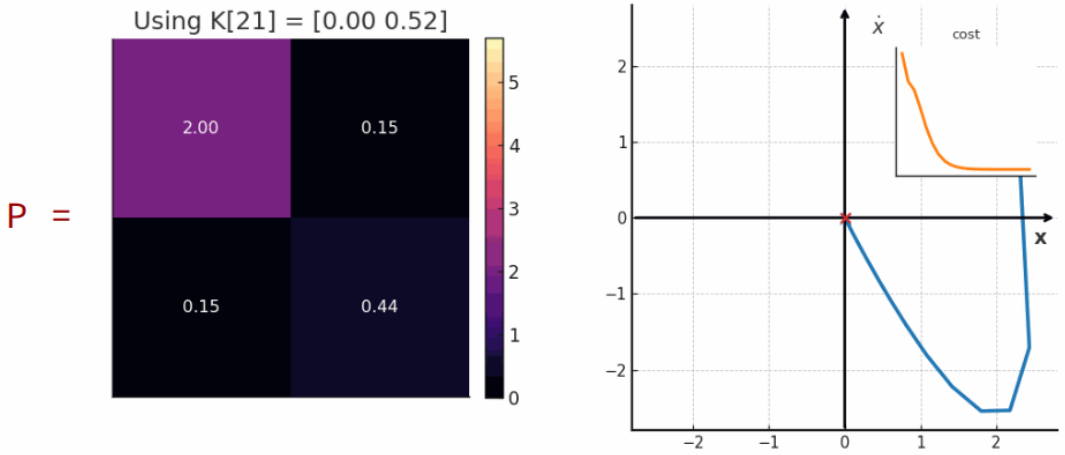


Figure 7: $P[h]$ as a heat map, the current gain $K[h]$ and the state-space trajectory

Intuition

LQR is value iteration where the whole value table is replaced by a single matrix P_h. One backward pass of cheap matrix algebra gives the *optimal controller for every time and every state*, with no grid. It is a controller, not just a plan: $u = K_h x$ tells you what to do from *wherever you end up*, which makes it robust to disturbances. Note also that K_h does not depend on the initial state at all.

10 LQR Extensions *(slides 71–76)*

The extensions make LQR applicable to: affine systems, systems with stochasticity, regulation around a non-zero fixed point for non-linear systems, trajectory following for non-linear systems, ...

10.1 LQR for Linear Time Varying (LTV) Systems *(slides 72–73)*

LTV system: $x_{t+1} = F_{x,t}x_t + F_{u,t}u_t$, cost $c(x_t, u_t) = x_t^\top Q_t x_t + u_t^\top R_t u_t$ (so F_x, F_u, C_x, C_u all carry the index t). Going backwards ($i = 1, 2, \dots$) the same recursion holds with time-indexed matrices:

$$K^*_{H-i} = -\left(C_{u,H-i} + F_{u,H-i}^\top P_{H-i+1} F_{u,H-i}\right)^{-1} F_{u,H-i}^\top P_{H-i+1} F_{x,H-i},$$
$$P_{H-i} = C_{x,H-i} + K^{*\top}_{H-i} C_{u,H-i} K^*_{H-i} + (F_{x,H-i} + F_{u,H-i}K^*_{H-i})^\top P_{H-i+1}(\cdots).$$

Intuition

The proof of Theorem 8.1 never used that the matrices were the same at every step; it only used one step at a time. So the same argument goes through with time-indexed matrices. This is the engine for trajectory following and iLQR below.

10.2 LQR for Affine Systems *(slides 74)*

Affine system $x_{t+1} = F_x x_t + F_u u_t + c$ with cost $x_t^\top C_x x_t + u_t^\top C_u u_t$. Re-define the state and transition by appending a constant coordinate, $z_t = [x_t; 1]$:

$$z_{t+1} = \begin{bmatrix} x_{t+1} \\ 1 \end{bmatrix} = \begin{bmatrix} F_x & c \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ 1 \end{bmatrix} + \begin{bmatrix} F_u \\ 0 \end{bmatrix} u_t = F'_x z_t + F'_u u_t,$$

and the cost on z is $z^\top \mathrm{diag}(C_x, 0) z + u^\top C_u u$ (positive *semi*-definite, which Theorem 8.1 allows — this is why the companion notes only ask $C_x \succeq 0$). The resulting policy is affine in x: $u = K_{x,h}x + k_h$.

Intuition

The constant offset c is a "bias" in the dynamics. By pretending the number 1 is just another state that never changes, the bias becomes an ordinary linear term, and the linear machinery applies unchanged.

10.3 LQR for Stochastic Systems *(slides 75)*

Stochastic system $x_{t+1} = F_x x_t + F_u u_t + w_t$, $w_t \sim \mathcal{N}(0, \sigma^2 I)$, cost as before. The optimal control policy is the *same*; the cost-to-go has an extra term that depends on the variance of the system and cannot be controlled: $J^*_h = x_t^\top P_h x_t + p_h$, with base case $p_H = 0$ and $p_h = \mathrm{tr}(\sigma^2 P_{h+1}) + p_{h+1}$.

Proposition 10.1 (Certainty equivalence). *The optimal gains K_h are exactly those of Theorem 8.1, since the noise does not change the control law, and the value acquires an additive constant, $V_h^*(x) = x^\top P_h x + \sum_{\tau>h} \operatorname{tr}(P_\tau \Sigma)$.*

Proof. In the induction step the only change is the expectation over w:

$$\mathbb{E}\big[(F_x x + F_u u + w)^\top P_{h+1}(F_x x + F_u u + w)\big] = (F_x x + F_u u)^\top P_{h+1}(F_x x + F_u u) + \operatorname{tr}(P_{h+1}\Sigma),$$

the cross terms vanishing because $\mathbb{E}[w] = 0$ and w is independent of (x, u). The added term does not depend on u, so the minimiser is unchanged; it accumulates through the recursion as stated. □

Intuition

Zero-mean noise pushes you around but, on average, it has no preference for any direction, so the best *response* to the state you see is unchanged: act as if there were no noise ("certainty equivalence"). The noise only costs you a fixed "tax" $\operatorname{tr}(P\Sigma)$ at every step that you cannot avoid. Example: with $\Sigma = \sigma^2 I$ and $P_{h+1} = 1.5$ in one dimension, each remaining step adds $1.5\sigma^2$ to the expected cost.

10.4 Penalize for Change in Controls *(slides 76)*

To penalise changes in the control rather than its magnitude, augment the state with the previous control, $x'_t = [x_t;\, u_{t-1}]$, and take $\Delta u_t = u_t - u_{t-1}$ as the new control:

$$\begin{bmatrix} x_{t+1} \\ u_t \end{bmatrix} = \begin{bmatrix} F_x & F_u \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ u_{t-1} \end{bmatrix} + \begin{bmatrix} F_u \\ I \end{bmatrix} \Delta u_t, \qquad \text{cost} = x'^\top C'_x x' + \Delta u^\top C'_u \Delta u, \quad C'_x = \begin{bmatrix} C_x & 0 \\ 0 & C_u \end{bmatrix}.$$

(Check: $x_{t+1} = F_x x_t + F_u(u_{t-1} + \Delta u_t) = F_x x_t + F_u u_t$.)

Intuition

Real actuators cannot jump: a car cannot change its steering angle instantly without wearing the hardware and unsettling the passengers. By making "the control I applied last time" part of the state, the *change* in control becomes the thing you choose and pay for, and you again have a standard LQR problem.

11 LQR for Non-Linear Systems *(slides 77–82)*

Non-linear system $x_{t+1} = f(x_t, u_t)$, cost $x_t^\top C_x x_t + u_t^\top C_u u_t$. We have a goal (x^*, u^*) that is a fixed point, $x^* = f(x^*, u^*)$, and want to stabilise the system around it.

Linearisation (slides 78–80). First-order Taylor expansion around (x^*, u^*):

$$x_{t+1} \approx f(x^*, u^*) + \frac{\partial f}{\partial x}(x^*, u^*)(x_t - x^*) + \frac{\partial f}{\partial u}(x^*, u^*)(u_t - u^*).$$

Let $z_t = x_t - x^*$ and $v_t = u_t - u^*$. Since $f(x^*, u^*) = x^*$,

$$z_{t+1} = A\, z_t + B\, v_t, \qquad A = \frac{\partial f}{\partial x}(x^*, u^*),\ B = \frac{\partial f}{\partial u}(x^*, u^*),$$

a linear system with cost $z^\top C_x z + v^\top C_u v$. **Solve it the usual way (slide 81):** $v_t = K z_t \Rightarrow u_t = u^* + K(x_t - x^*)$.

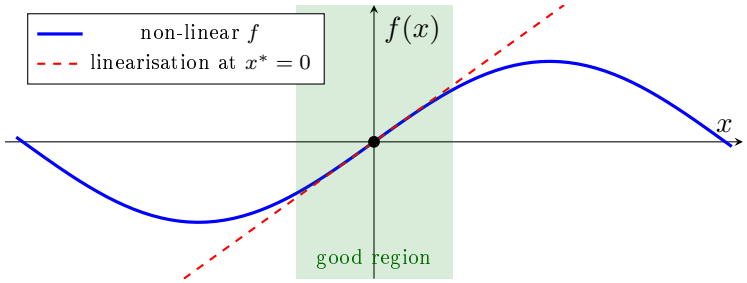


Figure 8: Original illustration (not on slides): **Near the fixed point the tangent is a good model of f (shaded), far from it it is not.**

Linearising the cost (slide 82). We can also linearise the cost with a 2nd-order Taylor expansion ("do it at home"), which gives something like

$$x_t^\top C_x x_t + u_t^\top C_u u_t + u_t^\top M x_t + x_t^\top q + u_t^\top r + c,$$

i.e. a quadratic with a *cross term* M, linear terms q, r and a constant c. The linear terms are handled by the constant-coordinate trick, the cross term by the cross-term form of Theorem 8.1.

Intuition

Zoom in enough on any smooth curve and it looks like a straight line. LQR on the linearised system is optimal for the *linear* approximation; it is a good controller as long as the state stays close to the point you linearised around — which is what a regulator tries to do. Think of balancing a pole: you only care about small deviations from upright.

12 LQR for Trajectory Following in Non-Linear Systems *(slides 83–85)*

Non-linear system $x_{t+1} = f(x_t, u_t)$; given a trajectory $x_0^*, \dots, x_T^*$ we want controls such that $x_{t+1}^* = f(x_t^*, u_t^*)$ along the trajectory; in other words we solve (slide 84)

$$\min_{u_0,\dots,u_{H-1}} \sum_{t=0}^{H-1} (x_t - x_t^*)^\top C_x (x_t - x_t^*) + (u_t - u_t^*)^\top C_u (u_t - u_t^*) \quad \text{s.t. } x_{t+1} = f(x_t, u_t).$$

We use the same trick, but linearise around (x_t^*, u_t^*) — the linearisation point *changes with* t, so we get an LTV system (slide 85): with $z_t = x_t - x_t^*$ and $v_t = u_t - u_t^*$,

$$z_{t+1} \approx A_t z_t + B_t v_t, \quad A_t = \frac{\partial f}{\partial x}(x_t^*, u_t^*),\ B_t = \frac{\partial f}{\partial u}(x_t^*, u_t^*),$$

which is solved with LQR for LTV systems, giving $u_t = u_t^* + K_t(x_t - x_t^*)$.

Intuition

Regulating around a point is the special case where the target never moves. Following a trajectory is regulating around a *moving* point: at each time you linearise the world around where you *should* be, and compute the gain that pulls you back onto the path. This is the idea you need for iLQR: if the trajectory you are tracking is the one you are currently trying to improve, you get a sequence of better and better LQR problems.

13 Iterative LQR (iLQR) *(slides 86–92)*

13.1 Iterative LQR (iLQR) *(slides 86)*

iLQR attempts to solve the *most general case* $\min \sum_{h=0}^{H} g_h(x_h, u_h)$ subject to $x_{t+1} = f(x_t, u_t)$, by iteratively approximating it and leveraging the LQR formulation.

13.2 Iterative LQR — Algorithm *(slides 87)*

Algorithm 13.1 (Iterative LQR). *Initialisation:* pick either a control policy $\pi^{(0)}$ or a sequence $\{\bar{x}_t^{(0)}, \bar{u}_t^{(0)}\}_{t=0}^{H}$ (by simulating any policy or by guessing). For $i = 0, \dots, I$:

1. execute (or simulate) $\pi^{(i)}$ and collect $\{\bar{x}_t^{(i)}, \bar{u}_t^{(i)}\}_{t=0}^{H}$;
2. obtain the LQ approximation of the problem, linearising around $\{\bar{x}_t^{(i)}, \bar{u}_t^{(i)}\}$ with a 1st-order Taylor expansion for the dynamics and a 2nd-order one for the cost, giving a time-varying linear-quadratic problem in the deviations;
3. compute $\pi^{(i+1)}$ with the LQR-LTV formulation, in its cross-term form and augmenting the state with a constant coordinate as for affine dynamics: the expanded cost has a linear term wherever its gradient along the reference is not zero, the constant coordinate turns its part in δu_t into a mixed term, and its Hessian may couple δx_t and δu_t as well. The solution is an *affine* policy $\delta u_t = k_t^{(i)} + K_t^{(i)} \delta x_t$;
4. roll the true system forward under $u_t = \bar{u}_t^{(i)} + k_t^{(i)} + K_t^{(i)}(x_t - \bar{x}_t^{(i)})$ to obtain the next trajectory, and iterate.

13.3 Iterative LQR — LTV *(slides 88–89)*

With $z_t = [x_t - \bar{x}_t^{(i)};\ 1]^\top$ and $v_t = u_t - \bar{u}_t^{(i)}$ the dynamics are

$$F_{x,t} = \begin{bmatrix} \frac{\partial f}{\partial x}(\bar{x}_t^{(i)}, \bar{u}_t^{(i)}) & f(\bar{x}_t^{(i)}, \bar{u}_t^{(i)}) - \bar{x}_{t+1}^{(i)} \\ 0 & 1 \end{bmatrix}, \qquad F_{u,t} = \begin{bmatrix} \frac{\partial f}{\partial u}(\bar{x}_t^{(i)}, \bar{u}_t^{(i)}) \\ 0 \end{bmatrix},$$

(the top-right block is zero when the reference is a true rollout), and, assuming $g(x, u) = g(x) + g(u)$ (separable), the 2nd-order expansion of the cost $g(x_t, u_t) \approx z_t^\top C_{x,t} z_t + v_t^\top C_{u,t} v_t + 2 z_t^\top N_t v_t$ has

$$C_{x,t} = \begin{bmatrix} \frac{1}{2}\frac{\partial^2 g}{\partial x^2} & \frac{1}{2}\frac{\partial g}{\partial x} \\ \frac{1}{2}\left(\frac{\partial g}{\partial x}\right)^\top & g(x) + g(u) \end{bmatrix}, \quad C_{u,t} = \frac{1}{2}\frac{\partial^2 g}{\partial u^2}, \quad N_t = \begin{bmatrix} 0 \\ \frac{1}{2}\left(\frac{\partial g}{\partial u}\right)^\top \end{bmatrix},$$

all evaluated at $(\bar{x}_t^{(i)}, \bar{u}_t^{(i)})$. This is exactly the *quadratic approximation behind iLQR*: its $2z^\top N v$ term is the mixed term mentioned in step 3 of Algorithm 13.1. Check: expanding, $z^\top C_x z = \frac{1}{2}\delta x^\top g_{xx} \delta x + g_x^\top \delta x + g$, $v^\top C_u v = \frac{1}{2} v^\top g_{uu} v$, $2 z^\top N v = g_u^\top v$, which is the second-order Taylor expansion of g.

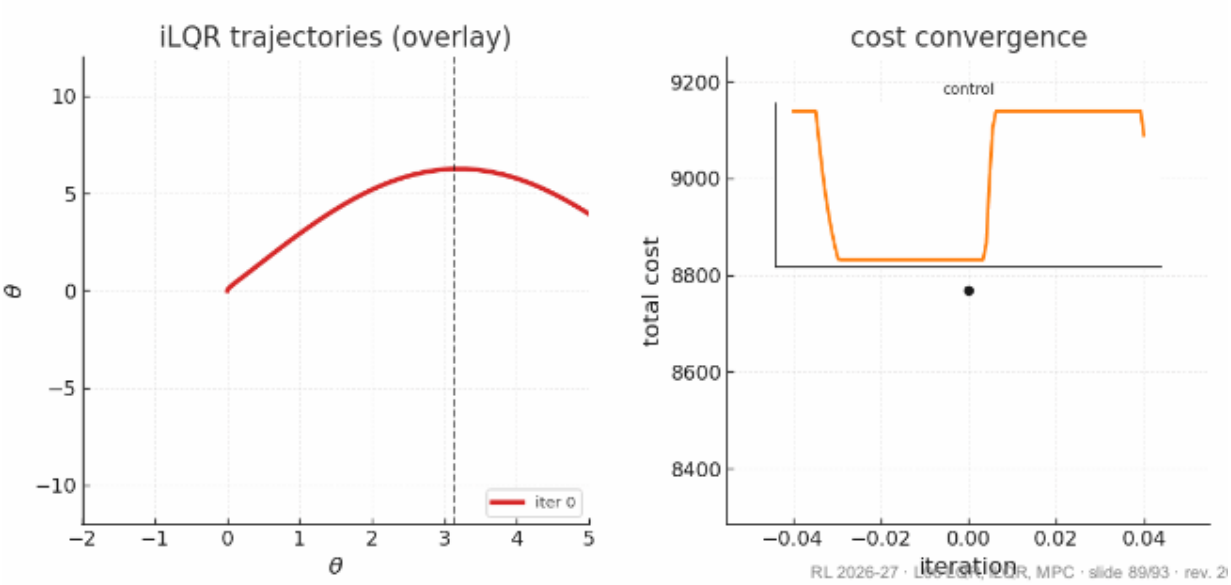


Figure 9: left panel "iLQR trajectories (overlay)" in the $(\theta, \dot{\theta})$ plane, right panel "cost convergence"

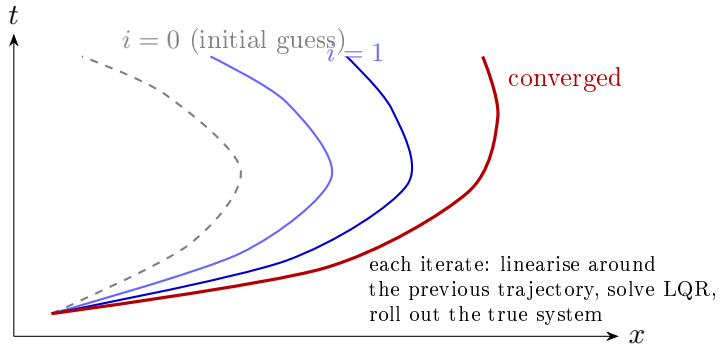


Figure 10: Original schematic (not on the slides) of iLQR: each iteration replaces the nominal trajectory with the roll-out of the LQR policy of the local LQ model.

Intuition

iLQR is Newton-like optimisation done by dynamic programming. At each iteration you build a *local* linear-quadratic model of the world around your current best trajectory, solve it exactly with LQR, and apply the result to the real system to get a new (better) trajectory. The affine policy $\delta u = k + K\delta x$ has two parts: k is the feed-forward correction to the plan, K is the feedback that keeps you on it.

13.4 Iterative LQR — Convergence *(slides 90–91)*

As it is, iLQR might not converge: the optimal policy of the LQ approximation might end up *not staying close* to the linearisation points. **Solution:** change the cost to

$$(1-\alpha)\, g(x_t, u_t) + \alpha\big(\|x_t - \bar{x}_t^{(i)}\|_2^2 + \|u_t - \bar{u}_t^{(i)}\|_2^2\big),$$

and if g is bounded and α is close enough to 1, the second term dominates and ensures the linearisations are good approximations of the solution trajectory.

Intuition

It is a leash. The LQ model is only trustworthy near the nominal trajectory, so you add a penalty for straying from it. With $\alpha = 0.9$, one unit of deviation costs 0.9 while a unit of true cost counts 0.1: the update becomes timid but safe, and you take many small, reliable steps instead of one big wrong leap. This is the same idea as a trust region.

13.5 Iterative LQR — Tricks *(slides 92)*

- **Initialisation matters**: you can get stuck in local optima. Alternatives: simulated annealing (if there are no computation constraints) or line search.
- If g is non-convex, the LQ approximation may give non positive-definite C_x, C_u. **Fix:** add more cost for deviating from the linearisation points until they become positive-definite.

14 Receding Horizon Control, i.e. Model Predictive Control (MPC) *(slides 93)*

At iLQR convergence, the system could execute the policy and still not be at the expected trajectory (inaccurate dynamics, noise, ...). **Idea:** at every time step t replan from t to H. **In practice:** replanning until H at every step is too costly, so replan over a *shorter horizon* h and use a cost-to-go $J(t+h)$ at its end. This is a Model Predictive Controller with an iLQR solver.

Algorithm 14.1 (Receding-horizon control (MPC))**.** At each time t: solve the control problem from the current *measured* state over a short horizon $[t, t+h]$, with a terminal cost approximating the cost-to-go beyond h; apply only u_t; discard the rest; measure; repeat.

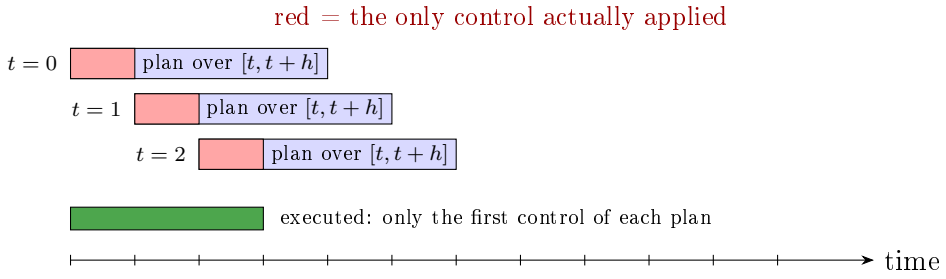


Figure 11: Original schematic (not on the slides): the planning window slides forward at every step.

Intuition

It is the way you navigate with GPS: you do not drive the original route blindly, you recompute from where you *are* after every turn. Replanning gives feedback through the optimiser, so errors due to noise or a wrong model are corrected at the next step. The finite window keeps the computation affordable, and the terminal cost-to-go $J(t+h)$ stops the controller from being short-sighted.

Method	Setting	Output	Cost / scaling	Main limitation
Backward induction (Thm. 3.2)	finite-horizon MDP, discrete S, A	V_h^*, π_h^* (tables), exact	$O(H\|S\|^2\|A\|)$; with a grid $\Theta((1/\epsilon)^{2d+k})$	curse of dimensionality: only $d \lesssim 5$–6
LQR (Thm. 8.1)	linear dynamics, quadratic cost	K_h, P_h, optimal linear feedback, exact	closed-form Riccati recursion, polynomial in d, k (matrix products/inverse)	needs linear model and quadratic cost
LQR extensions	LTV, affine, stochastic (Prop. 10.1), Δu penalty	same recursion on augmented/time-indexed matrices	same as LQR	still linear-quadratic
Linearised LQR	non-linear f near a fixed point or a reference trajectory	$u = u^* + K(x - x^*)$	LQR-LTV along reference	valid only near the reference
iLQR (Alg. 13.1)	general non-linear f, general g	locally optimal affine policy / trajectory	$I\times$ (linearise + LQR-LTV)	may diverge; local optima; needs trust-region/penalty
MPC (Alg. 14.1)	any of the above, online	u_t re-computed at every step	one short-horizon solve per step	model error, compute budget per step; needs terminal cost

Table 2: Summary of the methods of lecture L06.

15 Summary

15.1 Comparison of the methods

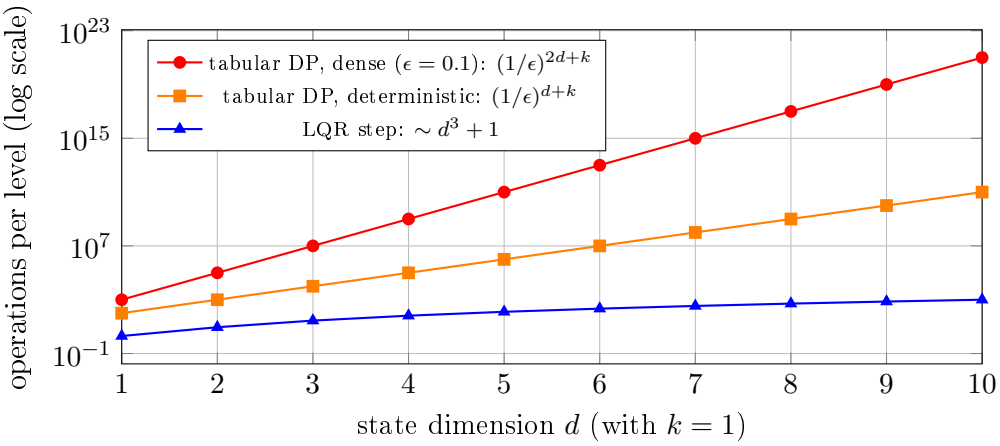


Figure 12: Qualitative comparison (original, order-of-magnitude, constants ignored): tabular DP grows exponentially in d, a Riccati step grows like a polynomial. LQR escapes the curse of dimensionality by *structure* (linear dynamics, quadratic cost), not by approximation.

15.2 Key take-aways

- **Finite horizon makes DP exact and one-pass**: backward induction from the deadline gives time-dependent optimal policies, but its cost explodes with dimension when the state

is discretised (Bellman's curse).

- **LQR is value iteration in closed form**: for linear dynamics and quadratic cost the value is $x^\top P_h x$ and the policy is the linear feedback $u = K_h x$, computed by the Riccati recursion with no discretisation.
- **Extensions reduce to the same recursion**: affine terms (constant coordinate), LTV matrices, penalties on Δu (augmented state); additive zero-mean noise leaves the gains unchanged (certainty equivalence) and only adds $\mathrm{tr}(P\Sigma)$ to the cost.
- **Non-linear problems are handled by repeated local LQR**: linearise around a point or a trajectory (regulation, tracking), and iterate in iLQR with 1st-order dynamics and 2nd-order cost; keep the iterates close to the linearisation points (penalty weight α) and watch for local optima and non positive-definite Hessians.
- **MPC adds feedback through replanning**: solve over a short horizon from the measured state, apply the first control only, repeat; this makes the system robust to model error and noise at the price of one solve per time step.

A Supplementary material (in the companion PDFs, not on the slides)

The following result is in the companion notes "Finite-horizon MDPs and optimal control" but has **no corresponding slide**: the slides only state (slides 13–15) that policies are time-dependent, without an example proving it is necessary.

Proposition A.1 (The optimal policy depends on the time remaining)**.** *There are finite-horizon MDPs in which every optimal policy satisfies $\pi^*_h(s) \neq \pi^*_{h'}(s)$ for some state s and some $h \neq h'$.*

Proof. Two actions at a single state s. Action *safe* pays 1 and returns to s. Action *gamble* pays 0 now and moves to an absorbing state g paying 3 per step with probability 1/2, or to an absorbing state b paying 0 per step with probability 1/2. Write $n = H - h$ for the number of steps remaining and V_n for the optimal value at s with n steps left, $V_0 = 0$. Gambling now is worth $Q_n(\text{gamble}) = \frac{1}{2} \cdot 3(n-1) = 1.5(n-1)$, since the first step pays nothing and the remaining $n-1$ pay 3 half the time; playing safe now is worth $Q_n(\text{safe}) = 1 + V_{n-1}$, one now and the best play afterwards.

For $n \leq 3$ safe is optimal throughout: by induction $V_{n-1} = n-1$, so $Q_n(\text{safe}) = n$, which beats $1.5(n-1)$ at $n \leq 2$ and ties with it at $n = 3$, giving $V_3 = 3 = 1.5 \cdot 2$. From there $V_{n-1} = 1.5(n-2)$ by induction, and

$$Q_n(\text{gamble}) - Q_n(\text{safe}) = 1.5(n-1) - 1 - 1.5(n-2) = \tfrac{1}{2} > 0 \qquad (n \geq 4),$$

so the gamble is optimal and $V_n = 1.5(n-1)$, which carries the induction. So safe is strictly better for $n \leq 2$, the two tie at $n = 3$, and the gamble is strictly better for $n \geq 4$. Any optimal policy therefore gambles early in a long episode and plays safe in the last two steps. □

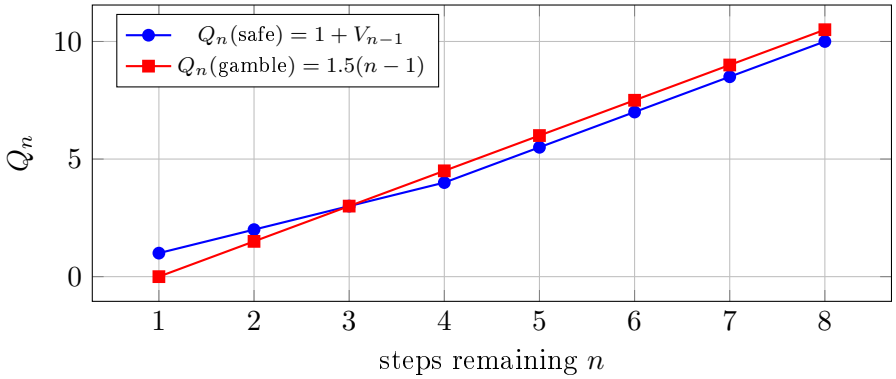


Figure 13: Values of the two actions in the proof of Proposition A.1 (original figure). The curves cross at $n = 3$.

Intuition

Gambling pays off only if there is enough time to harvest the jackpot: the expected gain per remaining step (1.5) beats the safe payoff (1) only if enough steps remain to compensate for the one step you give up. Concretely, with $n = 2$ you get 2 safe versus 1.5 from gambling, with $n = 4$ you get 4 versus 4.5. The identical state calls for different actions at different times, so no single stationary policy can be optimal; this justifies the time-dependent policies $\pi = (\pi_0, \ldots, \pi_{H-1})$ of Definition 2.1.

A note on the companion notes' "slides only" items. The self-check of the LQR PDF lists as "on the slides only" the derivations for affine dynamics, penalising changes in control, LQR around a fixed point, trajectory following, and the quadratic approximation behind iLQR. They are in the main text at their slide position (Sections on LQR Extensions, Non-Linear Systems, Trajectory Following, iLQR).